

Universidade Federal do Ceará
Campus Quixadá

QXD0068 – Reuso de Software

Prof. Francisco Victor da Silva Pinheiro

Trabalho 1
Refatoração com Padrões de Projeto em Sistema Legado

Alunos:

Daniel Christopher Souza Rodrigues - 552493
Francisco Gustavo Martins de Sousa - 511779

Quixadá-CE
2026

Sumário

1. Introdução
2. Caracterização e Diagnóstico do Sistema Legado
3. Padrões de Projeto Selecionados
4. Refatorações Realizadas
5. Análise dos Resultados
6. Conclusão
7. Referências

1. Introdução

O reuso de software envolve aproveitar estruturas e comportamentos existentes de modo que possam atender diferentes necessidades com menos repetição. Em um sistema em evolução, a distribuição de responsabilidades e o acoplamento a bibliotecas externas podem limitar esse aproveitamento. A refatoração permite reorganizar o código mantendo os comportamentos avaliados, enquanto os padrões de projeto oferecem formas conhecidas de organizar essa colaboração.

O sistema selecionado é o Homarr (legado), um dashboard que reúne aplicações e widgets em áreas configuráveis e consulta informações de serviços externos. Sua interface utiliza React e seu código é predominantemente TypeScript. O projeto foi escolhido pela presença de funcionalidades reais, integrações externas e, por ser um software legado, problemas de manutenção identificáveis.

O objetivo do trabalho é identificar problemas de projeto, justificar a aplicação de pelo menos quatro padrões GoF, refatorar o código e reunir evidências de preservação dos comportamentos afetados. O código comparável está em https://github.com/odanielchristopher/reuso-trabalho-I. A pasta homarr preserva a referência original e homarr-refactor concentra as alterações.

1.1 Procedimento adotado

A análise foi seguida por testes de caracterização executados antes das refatorações. Posteriormente, os mesmos cenários e expectativas foram mantidos, complementados por verificação de tipos, build e observações de funcionamento. A comparação separa problemas anteriores, alterações estruturais, benefícios observados e limites ainda pendentes.

2. Caracterização e Diagnóstico do Sistema Legado

2.1 Caracterização do Sistema

A versão analisada é Homarr 0.16.0, conforme o diagnóstico local. O sistema permite organizar apps e widgets em wrappers, categorias e barras laterais, definir posição e dimensões por tamanho de tela, editar e salvar boards e escolher ícones de fontes locais ou remotas. Também possui autenticação e integrações com outros serviços.

As tecnologias registradas incluem TypeScript 5.1.6, React 18, Next.js 13.5.11, Yarn 3.6.0, Vitest 0.33.0 e happy-dom. O projeto contém frontend e código de servidor, com rotas e clientes externos. Os testes de componentes utilizam Testing Library. As verificações documentadas empregaram Node.js 22.23.3.

2.2 Problemas Identificados

2.2.1 Conhecimento de GridStack distribuído entre arquivos

initializeGridstack já centralizava parte da inicialização. Entretanto, useGridstack mantinha a instância GridStack, ajustava dimensões e liberava eventos. O Trecho 1 mostra chamadas diretas à biblioteca e manipulação de CSS no hook, que também selecionava itens e atualizava a configuração do board.

Trecho 1 — Operações de integração no hook. Fonte: homarr/src/components/Dashboard/Wrappers/gridstack/use-gridstack.ts.

```
const gridRef = useRef<GridStack>();

useEffect(() => {
  if (areaType === 'sidebar') return;
  const widgetWidth = mainAreaWidth / wrapperColumnCount;

  root.style.setProperty('--gridstack-widget-width', widgetWidth.toString());
  gridRef.current?.cellHeight(widgetWidth);
}, [mainAreaWidth, wrapperColumnCount, gridRef.current]);
```

O acoplamento também aparecia nos callbacks: onChange recebia GridStackNode e lia atributos DOM para identificar o item, conforme o Trecho 2. Desse modo, o código que atualizava o board precisava conhecer a representação de eventos da biblioteca externa.

Trecho 2 — Início do callback onChange. Fonte: homarr/src/components/Dashboard/Wrappers/gridstack/use-gridstack.ts.

```
const onChange = isEditMode
  ? (changedNode: GridStackNode) => {
      if (!configName) return;

      const itemType = changedNode.el?.getAttribute('data-type');
      const itemId = changedNode.el?.getAttribute('data-id');
      if (!itemType || !itemId) return;
```

No componente de apresentação WrapperContent, a referência gridstack era declarada nas propriedades sem ser utilizada. O Trecho 3 mostra essa dependência de tipo. Considerando o inicializador, o hook e WrapperContent, três arquivos de produção de Wrappers importavam a biblioteca. A evidência indica uma integração distribuída; não significa que toda dependência externa seja um defeito ou deva ser eliminada.

Trecho 3 — Referência GridStack no contrato de apresentação. Fonte: homarr/src/components/Dashboard/Wrappers/WrapperContent.tsx.

```
import { GridStack } from 'fily-publish-gridstack';

interface WrapperContentProps {
  apps: AppType[];
  widgets: IWidget<string, any>[];
  refs: {
    wrapper: RefObject<HTMLDivElement>;
    items: MutableRefObject<Record<string, RefObject<HTMLDivElement>>>;
    gridstack: MutableRefObject<GridStack | undefined>;
  };
}
```

2.2.2 Duplicação na inclusão de apps e widgets

O Trecho 4 apresenta dois percursos de initializeGridstack. Ambos obtêm o elemento, aplicam atributos de geometria, chamam makeWidget e capturam a posição atribuída pela biblioteca quando não existe shape salvo. A principal diferença é a identificação do tipo como app ou widget. Uma alteração nessa sequência precisaria ser reproduzida nos dois blocos.

Trecho 4 — Percursos repetidos de inclusão. Fonte: homarr/src/components/Dashboard/Wrappers/gridstack/init-gridstack.ts.

```
items.forEach(({ id, shape }) => {
  const item = itemRefs.current[id]?.current;
  setAttributesFromShape(item, shape[shapeSize]);
  item && grid.makeWidget(item as HTMLDivElement);
  if (!shape[shapeSize] && item) {
    const gridItemElement = item as GridItemHTMLElement;
    if (gridItemElement.gridstackNode) {
      const { x, y, w, h } = gridItemElement.gridstackNode;
      tilesWithUnknownLocation.push({ x, y, w, h, type: 'app', id });
    }
  }
});
widgets.forEach(({ id, shape }) => {
  const item = itemRefs.current[id]?.current;
  setAttributesFromShape(item, shape[shapeSize]);
  item && grid.makeWidget(item as HTMLDivElement);
  if (!shape[shapeSize] && item) {
    const gridItemElement = item as GridItemHTMLElement;
    if (gridItemElement.gridstackNode) {
      const { x, y, w, h } = gridItemElement.gridstackNode;
      tilesWithUnknownLocation.push({ x, y, w, h, type: 'widget', id });
    }
  }
});
```

setAttributesFromShape já compartilhava a definição de gs-x, gs-y, gs-w e gs-h. Portanto, a duplicação demonstrada é do fluxo que chama esse helper, inclui o item e recupera sua posição, não da implementação interna dos atributos. A oportunidade era compartilhar essa sequência e concentrar a integração, mantendo as decisões de configuração no hook.

2.2.3 Repetição do fluxo nas fontes remotas de ícones

AbstractIconRepository já compartilhava o tratamento de exceções. Entretanto, as fontes remotas ainda implementavam a consulta HTTP, a leitura de JSON e a montagem da resposta normalizada. Os Trechos 5 e 6 permitem comparar essa repetição em jsDelivr e unpkg. Foram omitidas as transformações específicas para destacar a sequência comum.

Trecho 5 — Fluxo da fonte jsDelivr. Fonte: homarr/src/tools/server/images/jsdelivr-icons-repository.ts. Transformação omitida.

```
protected async fetchInternally(): Promise<NormalizedIconRepositoryResult> {
  const response = await fetch(this.repository.api);
  const body = (await response.json()) as JsdelivrResponse;

  // Trecho omitido: filtros e transformação especificos de jsDelivr.

  return {
    entries: normalizedEntries,
    count: normalizedEntries.length,
    success: true,
    name: this.displayName,
    copyright: this.copyright,
  };
}
```

Trecho 6 — Fluxo da fonte unpkg. Fonte: homarr/src/tools/server/images/unpkg-icons-repository.ts. Transformação omitida.

```
protected async fetchInternally(): Promise<NormalizedIconRepositoryResult> {
  const response = await fetch(`${this.repository}?meta`);
  const body = (await response.json()) as UnpkgResponse;

  // Trecho omitido: filtros e transformação especificos de unpkg.

  return {
    entries: normalizedEntries,
    count: normalizedEntries.length,
    success: true,
    name: this.displayName,
    copyright: this.copyright,
  };
}
```

As URLs e os tipos de resposta variam, mas a consulta, a leitura do corpo e a montagem de entries, count, success, name e copyright seguem a mesma sequência. A classe GitHubIconsRepository apresentava estrutura semelhante. As particularidades de filtros, ordenação

e construção de URLs devem permanecer específicas de cada fonte. Já a fonte local utiliza filesystem, motivo pelo qual não deve receber um fluxo obrigatório de HTTP.

Os comentários que indicam recortes ou omissões foram acrescentados para a apresentação das evidências e não pertencem ao código original. Os trechos completos podem ser consultados na pasta homarr do repositório da dupla.

2.2.4 Conhecimento de clientes de download acoplado ao roteador

O roteador de downloads original concentrava em um único arquivo (download.ts) tanto a orquestração das requisições quanto toda a lógica de comunicação com cada cliente. A função GetDataFromClient continha um bloco switch com cinco casos, cada um instanciando diretamente o cliente externo, realizando a chamada e transformando a resposta. Essa estrutura exigia que o arquivo de rota conhecesse as APIs de Deluge, Transmission, QBittorrent, SABnzbd e NZBGet, além dos detalhes de conversão de unidades e mapeamento de campos de cada um.

No roteador de Usenet (usenet/router.ts), o mesmo padrão se repetia: cada procedimento (info, history, pause, resume, queue) duplicava internamente a construção do cliente NZBGet ou SABnzbd. A URL, as opções de conexão e o uso de callbacks com Promise eram reescritos em cinco lugares diferentes dentro do mesmo arquivo, com 394 linhas no total.

Trecho 7 — Bloco switch com criação direta de clientes. Fonte: homarr/src/server/api/routers/download.ts.

```
switch (app.integration?.type) {
  case 'deluge': {
    return reduceTorrent(
      await new Deluge({
        baseUrl: app.url,
        password: findField(app, 'password'),
      }).getAllData(),
    );
  }
  case 'sabnzbd': {
    const { origin } = new URL(app.url);
    const client = new Client(origin, findField(app, 'apiKey') ?? '');
    const queue = await client.queue();
    // ... conversão manual de unidades e mapeamento inline ...
  }
  // ... demais casos: transmission, qBittorrent, nzbGet
}
```

Cada novo cliente de download exigiria modificar GetDataFromClient e o switch, aumentando a responsabilidade de um único arquivo. A duplicação de construção de cliente no roteador de Usenet agravava o problema: uma mudança nas opções de conexão do NZBGet precisaria ser reproduzida manualmente nos cinco procedimentos.

3. Padrões de Projeto Selecionados

3.1 Facade

O Facade é um padrão estrutural que oferece uma interface simplificada para um subsistema. No Homarr, sua aplicação foi motivada pela coordenação de GridStack distribuída entre arquivos. DashboardLayoutFacade reúne inicialização, dimensões, inclusão de itens e assinatura/liberação de eventos. O cliente useGridstack mantém o ciclo de vida React e a atualização da configuração.

O resultado esperado era reduzir o conhecimento direto da biblioteca nos clientes e reunir operações que dependem da mesma instância. A alternativa de extrair funções de dimensões e cleanup foi considerada, contudo ela ainda manteria a instância e seus tipos expostos no hook. O custo da solução escolhida é uma classe adicional, contratos próprios e a montagem de descrições dos itens.

3.2 Template Method

O Template Method é um padrão comportamental no qual um método estabelece a sequência de um algoritmo e delega etapas variáveis às subclasses. O Homarr já utilizava esse padrão com um classe abstrata chamada AbstractIconRepository, a mudança que fizemos foi de ampliar essa colaboração por meio de RemoteIconsRepository, compartilhando o fluxo HTTP e a montagem do resultado entre as três fontes remotas que o Homarr possui.

As subclasses GitHubIconsRepository, JsdelivrIconsRepository e UnpkgIconsRepository implementam normalizeEntries. O resultado esperado era reutilizar a sequência invariável sem eliminar filtros e regras específicos. A base intermediária evita introduzir HTTP no repositório local. O custo é um nível adicional de herança e um contrato genérico, que descreve o corpo esperado em TypeScript sem validá-lo em execução.

3.3 Strategy

O Strategy é um padrão comportamental que define uma família de algoritmos, encapsula cada um deles e os torna intercambiáveis. No Homarr, sua aplicação foi motivada pelo acoplamento identificado em GetDataFromClient: a lógica de comunicação com cada cliente de download (Deluge, Transmission, QBittorrent, SABnzbd e NZBGet) estava misturada à orquestração do roteador, tornando impossível substituir ou acrescentar um cliente sem modificar o arquivo de rota.

A refatoração extraiu as estratégias para download-strategies.ts, definindo o tipo DownloadStrategy como uma função que recebe um ConfigAppType e retorna uma Promise de NormalizedDownloadAppStat. O mapa downloadStrategies associa cada IntegrationType à sua estratégia correspondente. O roteador passou a consultar esse mapa e delegar a execução, sem conhecer a implementação de nenhum cliente.

As classes ou componentes envolvidos são: download.ts (contexto, que seleciona e invoca a estratégia), download-strategies.ts (família de estratégias concretas), createTorrentStrategy (fábrica que parametriza estratégias de torrent), createSabnzbdAdapter e createNzbgetAdapter (adaptadores consumidos pelas estratégias de Usenet). O resultado esperado era reduzir a responsabilidade do roteador e permitir adicionar novos clientes sem modificar a rota.

3.4 Adapter

O Adapter é um padrão estrutural que converte a interface de uma classe em outra esperada pelos clientes, permitindo que classes com interfaces incompatíveis colaborem. No Homarr, sua aplicação foi motivada pela duplicação da construção e uso dos clientes SABnzbd e NZBGet no roteador de Usenet: cada procedimento (info, history, pause, resume, queue) replicava a inicialização do cliente e a conversão dos callbacks para Promise.

A refatoração introduziu usenet-adapters.ts com duas funções de fábrica: createSabnzbdAdapter e createNzbgetAdapter. Cada uma encapsula a criação do cliente correspondente e expõe métodos com interface uniforme (getInfo, getHistory, getQueue, pause, resume). Os clientes externos possuem APIs distintas — SABnzbd usa promessas nativas e NZBGet usa callbacks —, e os adaptadores absorvem essa diferença, fornecendo ao roteador e às estratégias de download uma interface comum.

Os componentes envolvidos são: usenet-adapters.ts (adaptadores), usenet/router.ts (cliente dos adaptadores, sem conhecimento direto dos clientes externos), download-strategies.ts (que também consome os adaptadores para SABnzbd e NZBGet). O resultado esperado era concentrar a conversão de interface e eliminar a repetição da inicialização de clientes nos procedimentos do roteador.

4. Refatorações Realizadas

4.1 Facade na integração GridStack

A estrutura anterior continha um inicializador útil, mas deixava dimensões e liberação de eventos no hook. A refatoração incorporou init-gridstack.ts à fachada e retirou a referência pública GridStack do retorno do hook. WrapperContent passou a declarar apenas as referências DOM efetivamente utilizadas.

4.1.1 Coordenação de dimensões antes e depois

Antes, o hook executava diretamente as operações da biblioteca e do CSS:

```
useEffect(() => {
  if (areaType === 'sidebar') return;
  const widgetWidth = mainAreaWidth / wrapperColumnCount;
  // widget width is used to define sizes of gridstack items within global.scss
  root.style.setProperty('--gridstack-widget-width', widgetWidth.toString());
  gridRef.current?.cellHeight(widgetWidth);
}, [mainAreaWidth, wrapperColumnCount, gridRef.current]);
```

Depois, o useEffect do React solicita as operações à fachada:

```
useEffect(() => {
  layout.updateDimensions(areaType, mainAreaWidth, wrapperColumnCount);
}, [mainAreaWidth, wrapperColumnCount, layout.initialized]);
```

O getter initialized informa a existência da instância; sua mudança não provoca renderização React. A execução do efeito continua dependendo das renderizações e dependências observadas. A refatoração não criou um mecanismo novo de atualização reativa.

4.1.2 Eventos e inclusão de itens

Os eventos change e added passaram a produzir LayoutItemChange, contendo identificação e geometria. O hook deixou de interpretar GridStackNode e atributos do elemento. Apps e widgets são descritos por id, type, shape e element e incluídos por um único percurso. A fachada retorna posições atribuídas aos itens sem shape salvo, e o hook decide como incorporá-las à configuração.

- **useGridstack**: Fornece itens e opções; chama a fachada nos efeitos; aplica alterações ao board
- **DashboardLayoutFacade**: Guarda GridStack privado; coordena initialize, updateDimensions e releaseEvents
- **GridStack** e **DOM**: Executam layout, eventos e atributos de geometria
- **WrapperContent**: Renderiza apps/widgets com referências DOM, sem declarar GridStack

4.1.3 Fronteira e preservação do comportamento

A tradução dos eventos é concentrada em forwardFirstItem. O recorte abaixo mantém o processamento somente do primeiro nó, conforme o comportamento anterior:

```
const node = nodes?.at(0);
if (!node) return;
callback({
  itemId: node.el?.getAttribute('data-id'),
  itemType: node.el?.getAttribute('data-type'),
  x: node.x,
  y: node.y,
  w: node.w,
  h: node.h,
});
```

releaseEvents remove as assinaturas change e added, mas não chama destroy. Essa escolha preserva o gerenciamento anterior; alterar a destruição da instância e sua relação com DOM gerenciado por React seria uma mudança adicional. Também foram mantidas as regras de configuração, regeneração e salvamento, externas à fachada.

4.1.4 Validação e evidências registradas

Os dois arquivos de caracterização permaneceram iguais aos anteriores, incluindo cenários e expectativas. A suíte posterior aprovou 126 testes em 16 arquivos, dos quais 29 verificam layout, hook e store. O typecheck apresentou saída idêntica à linha de base, com 15 diagnósticos anteriores. O build foi concluído, com avisos; sua configuração pula lint, cujo problema anterior permaneceu.

Os resultados estão em docs/reuso/evidencias/facade-suite-completa.log, facade-typecheck.log, facade-build.log, facade-testes-imutaveis.sha256 e facade-browser.json.

4.2 Template Method nas fontes de ícones

A estrutura anterior já compartilhava o tratamento de exceções em AbstractIconRepository, mas repetia consulta HTTP, leitura de JSON e montagem do resultado nas fontes remotas. A refatoração introduziu RemoteIconsRepository para definir essa sequência comum, mantendo nas subclasses a normalização específica de cada fonte. LocalIconsRepository permaneceu vinculada à base original, preservando seu fluxo de acesso ao filesystem.

4.2.1 Algoritmo antes e depois

Antes da refatoração, GitHub, jsDelivr e unpkg implementavam fetchInternally com uma sequência semelhante: consultar a fonte, interpretar o corpo JSON, transformar as entradas e montar a resposta normalizada. O trecho da implementação original de jsDelivr exemplifica essa organização. A consulta e a montagem do resultado estavam no mesmo método que os filtros e a transformação dos arquivos.

```
protected async fetchInternally(): Promise<NormalizedIconRepositoryResult> {
  const response = await fetch(this.repository.api);
  const body = (await response.json()) as JsdelivrResponse;

  const normalizedEntries = body.files
    .filter((file) => !['_banner.png', '_logo.png'].some((x) => file.name.includes(x)))
    .filter((file) => ['.png', '.svg'].some((x) => file.name.endsWith(x)))

    .map((file): NormalizedIcon => {
      const fileNameParts = file.name.split('/');
      const fileName = fileNameParts[fileNameParts.length - 1];
      const extensions = fileName.split('.')[1];
      return {
        url: this.repository.blob.replace('{0}', extensions).replace('{1}', fileName),
        name: fileName,
        size: file.size,
      };
    });

  return {
    entries: normalizedEntries,
    count: normalizedEntries.length,
    success: true,
    name: this.displayName,
    copyright: this.copyright,
  };
}
```

Depois, essa sequência passou para RemoteIconsRepository.fetchInternally. O método executa a requisição, lê o corpo, chama normalizeEntries e monta o resultado com as entradas e os metadados da fonte. As subclasses fornecem a etapa variável de normalização.

```
protected async fetchInternally(): Promise<NormalizedIconRepositoryResult> {
  const response = await (this.requestOptions === undefined
    ? fetch(this.requestUrl)
    : fetch(this.requestUrl, this.requestOptions));
  const body = (await response.json()) as TBody;
  const entries = this.normalizeEntries(body);

  return {
    entries,
    count: entries.length,
    success: true,
    name: this.displayName,
    copyright: this.copyright,
  };
}

protected abstract normalizeEntries(body: TBody): NormalizedIcon[];
```

Assim, a refatoração amplia o Template Method preexistente: além do tratamento de exceções compartilhado na base original, as fontes remotas passam a reutilizar o algoritmo de consulta e montagem do resultado.

4.2.2 Etapa variável e participantes

A etapa variável é representada por normalizeEntries, método abstrato protegido que recebe o corpo da resposta e devolve uma lista de NormalizedIcon. Cada subclasse interpreta o formato de sua fonte e aplica os filtros e transformações necessários. Na implementação refatorada de jsDelivr, o método contém somente esse processamento; a consulta HTTP e a montagem do resultado foram retiradas da subclasse.

A colaboração ocorre entre três níveis. AbstractIconRepository.fetch chama fetchInternally e trata as exceções. RemoteIconsRepository.fetchInternally define o algoritmo comum das fontes remotas. Por fim, GitHubIconsRepository, JsdelivrIconsRepository e UnpkgIconsRepository implementam normalizeEntries.

O uso demonstrado do padrão está na execução de um mesmo algoritmo herdado pelas três fontes, com uma etapa específica implementada por cada subclasse.

```
protected normalizeEntries(body: JsdelivrResponse): NormalizedIcon[] {
  return body.files
    .filter((file) => !['_banner.png', '_logo.png'].some((x) => file.name.includes(x)))
    .filter((file) => ['.png', '.svg'].some((x) => file.name.endsWith(x)))

    .map((file): NormalizedIcon => {
      const fileNameParts = file.name.split('/');
      const fileName = fileNameParts[fileNameParts.length - 1];
      const extensions = fileName.split('.')[1];
      return {
        url: this.repository.blob.replace('{0}', extensions).replace('{1}', fileName),
        name: fileName,
        size: file.size,
      };
    });
}
```

4.2.3 Fronteira e preservação do comportamento

A base intermediária foi restrita às fontes remotas porque a consulta HTTP e a leitura de JSON não pertencem ao fluxo da fonte local. LocalIconsRepository permaneceu diretamente vinculada a AbstractIconRepository, sem receber operações de rede desnecessárias.

As particularidades das fontes remotas foram preservadas. GitHub manteve seus filtros, a prioridade de SVG sobre PNG, o tamanho padrão zero e as opções da requisição. jsDelivr manteve seus filtros, a ordem das entradas e a transformação das URLs. unpkg conservou a consulta com ?meta e a seleção por type === 'file', inclusive para arquivos sem extensão de imagem.

O tratamento de exceções continuou em AbstractIconRepository. Não foram acrescentadas verificações de response.ok nem correções de formatos externos nesta etapa. Essas mudanças alterariam o comportamento e exigiriam análise e validação próprias.

4.2.4 Validação e evidências registradas

Os testes de caracterização foram mantidos sem alterações nas expectativas. Após a migração das três fontes, a suíte completa aprovou 126 testes em 16 arquivos, incluindo os 15 testes dos repositórios de ícones. Esses testes executam as classes reais com HTTP e filesystem simulados, verificando normalização, metadados e tratamento de falhas. O typecheck apresentou os mesmos 15 diagnósticos preexistentes, conforme os registros das etapas.

As evidências estão em docs/reuso/evidencias/template-method-suite-completa.log, template-method-typecheck.log, template-method-build-primeira.log, template-method-build-segunda.log e template-method-http-icons.json.

4.3 Strategy nas estratégias de download

A estrutura anterior concentrava em GetDataFromClient toda a lógica de comunicação com os cinco clientes de download. O switch case instanciava diretamente cada biblioteca externa e realizava a transformação da resposta dentro do próprio roteador, com 244 linhas no arquivo original.

4.3.1 Estrutura antes e depois

Antes, o roteador continha a função GetDataFromClient com a lógica de cada cliente:

```
// download.ts (original) — 244 linhas
const GetDataFromClient = async (app) => {
  const reduceTorrent = (data) => ({ type: 'torrent', appId: app.id, ... });
  switch (app.integration?.type) {
    case 'deluge':
      return reduceTorrent(await new Deluge({ baseUrl: app.url, ... }).getAllData());
    case 'sabnzbd':
      // instancia Client, chama queue(), converte unidades inline
    case 'nzbGet':
      // instancia NzbgetClient, usa callbacks com Promise, mapeia resposta
    // ... transmission, qBittorrent
  }
};
```

Depois, o roteador delega a execução ao mapa de estratégias:

```
// download.ts (refatorado) — 67 linhas
import { getDownloadDataFromClient } from './download/download-strategies';

// Na rota:
const response = await getDownloadDataFromClient(app);
```

Em download-strategies.ts, cada estratégia é declarada separadamente:

```
// download-strategies.ts
type DownloadStrategy = (app: ConfigAppType)
  => Promise<NormalizedDownloadAppStat | undefined>;

const downloadStrategies: Partial<Record<IntegrationType, DownloadStrategy>> = {
  deluge: createTorrentStrategy(Deluge, (app) => ({
    baseUrl: app.url, password: findAppProperty(app, 'password'),
  })),
  sabnzbd: async (app) => {
    const adapter = createSabnzbdAdapter(app, findAppProperty(app, 'apiKey'));
    const queue = await adapter.getQueue();
    return { type: 'usenet', appId: app.id, ... };
  },
  // ... demais estratégias
};

export const getDownloadDataFromClient = async (app) => {
  const strategy = app.integration?.type
    ? downloadStrategies[app.integration.type]
    : undefined;
  return strategy ? strategy(app) : undefined;
};
```

4.3.2 Participantes

O download.ts atua como contexto: seleciona a estratégia pelo tipo de integração e delega a execução, sem conhecer detalhes de nenhum cliente. O mapa downloadStrategies é a família de estratégias concretas, uma por tipo de integração. createTorrentStrategy é uma função de ordem superior que parametriza a criação de estratégias para clientes de torrent, eliminando a duplicação entre Deluge, Transmission e QBittorrent. As estratégias de Usenet consomem os adaptadores, mantendo a separação de responsabilidades.

4.3.3 Fronteira e preservação do comportamento

A normalização dos dados de torrent foi extraída para normalizeTorrentData, função pura que recebe apenas appId e os dados do cliente. A lógica de agregação de totais foi simplificada com reduce, substituindo os mapeamentos encadeados do original. A função getNzbgetState foi movida para usenet-adapters.ts como mapNzbgetState, preservando os três casos originais (QUEUED, PAUSED, downloading).

4.3.4 Validação e evidências registradas

Os testes em tests/refatoracao/routers/download-router.spec.ts verificam os seis cenários do roteador: normalização de torrents para Deluge com soma de totais, fila vazia com totais zerados, integração desconhecida omitida sem falhar a rota, manutenção de sucessos quando uma integração falha, SABnzbd com API key ausente, e NZBGet com cálculo de progresso, tamanho e ETA. Todos os testes aprovados confirmam que o comportamento anterior foi preservado na versão refatorada.

4.4 Adapter na integração com clientes Usenet

A estrutura anterior repetia a inicialização de NzbgetClient e a conversão de callbacks para Promise em todos os cinco procedimentos do roteador de Usenet. SABnzbd também era instanciado diretamente em cada procedimento, com conversão de unidades inline. O arquivo original totalizava 394 linhas.

4.4.1 Estrutura antes e depois

Antes, cada procedimento do roteador construía e usava o cliente diretamente. O trecho abaixo mostra a duplicação no procedimento queue — estrutura idêntica aparecia em info, history, pause e resume:

```
// usenet/router.ts (original) — procedimento queue, NZBGet
const url = new URL(app.url);
const options = {
  host: url.hostname,
  port: url.port || (url.protocol === 'https:' ? '443' : '80'),
  login: findAppProperty(app, 'username'),
  hash: findAppProperty(app, 'password'),
};
const nzbGet = NzbgetClient(options);
const nzbgetQueue = await new Promise((resolve, reject) => {
  nzbGet.listGroups((err, result) => {
    if (!err) resolve(result); else reject(err);
  });
});
```

Depois, os procedimentos delegam ao adaptador com uma linha por operação:

```
// usenet/router.ts (refatorado) — 194 linhas
if (app.integration.type === 'nzbGet') {
  const nzbgetQueue = await createNzbgetAdapter(app).getQueue();
  return { items: nzbgetQueue.items, total: nzbgetQueue.total };
}
return createSabnzbdAdapter(app, apiKey).getQueue(input.offset, input.limit);
```

O arquivo usenet-adapters.ts centraliza a construção dos clientes e expõe métodos com interface uniforme. A conversão de callbacks NZBGet para Promise foi encapsulada na função auxiliar promisify, reutilizada internamente pelos métodos do adaptador:

```
// usenet-adapters.ts — função auxiliar
const promisify = <T>(callback) =>
  new Promise<T>((resolve, reject) => callback(resolve, reject));

export const createNzbgetAdapter = (app) => {
  const nzbGet = createNzbgetClient(app);
  return {
    getInfo: async () => { /* usa promisify(nzbGet.status) */ },
    getHistory: async () => { /* usa promisify(nzbGet.history) */ },
    pause: () => promisify((resolve, reject) => {
      nzbGet.pauseDownload(false, (err, result) => ...);
    }),
    getQueue: async () => { /* promisify listGroups + status */ },
  };
};
```

4.4.2 Participantes

O usenet-adapters.ts define dois adaptadores: createSabnzbdAdapter e createNzbgetAdapter. Cada um recebe um ConfigAppType, constrói o cliente externo correspondente e expõe getInfo, getHistory, getQueue, pause e resume com tipos uniformes (UsenetInfo, UsenetHistory, UsenetQueue). usenet/router.ts é o cliente dos adaptadores e download-strategies.ts também os consume para os casos de Usenet, de modo que a interface adaptada é compartilhada entre dois contextos distintos.

4.4.3 Fronteira e preservação do comportamento

A conversão de unidades foi preservada nos adaptadores: SABnzbd converte kbpersec para bytes/s multiplicando por 1000, mbleft para bytes multiplicando por 1024^2, e o tempo restante com parseTimeLeft. NZBGet mantém RemainingSizeMB convertido para bytes ($\times 10^6$) e o cálculo de ETA por divisão pelo DownloadRate. As particularidades de mapeamento de cada cliente permaneceram no adaptador correspondente. A função mapSabQueueItem foi extraída como auxiliar para reutilização entre getQueue do SABnzbd e os slots da fila de downloads.

4.4.4 Validação e evidências registradas

Os testes em tests/refatoracao/routers/usenet-router.spec.ts cobrem oito cenários: informações do SABnzbd com conversão de tempo, tamanho e velocidade; histórico com paginação e noofslots; fila com conversão de itens; rejeição por API key ausente; pause e resume do NZBGet com argumento false; propagação de erro de callback; e rejeição de fila quando o resultado está ausente. A suíte confirma que o comportamento original foi preservado após a centralização nos adaptadores.

5. Análise dos Resultados

No Facade, os imports diretos de GridStack em arquivos de produção de Wrappers passaram de três para um. Essa contagem demonstra concentração local da dependência, não uma medida global de acoplamento. A fachada continua dependente de GridStack, DOM e ShapeType, e o hook depende de sua classe concreta.

A coesão da integração foi favorecida pelo agrupamento de operações que compartilham a mesma instância. A duplicação da inclusão de apps/widgets e da tradução dos eventos foi reduzida. Permanecem os dois mapeamentos que descrevem os itens e repetições de atualização da configuração no hook. O custo adicional é o contrato de itens/eventos e uma camada de coordenação.

No Template Method, a sequência HTTP, JSON e resultado passou a ser implementada uma vez e herdada por três fontes. As transformações específicas permaneceram nas subclasses. A fonte local mantém um fluxo coerente com filesystem. Isso demonstra reuso do algoritmo, mas acrescenta herança e exige cuidado para não abstrair diferenças legítimas.

As fronteiras deixam mais claro onde alterar a integração e onde manter regras do sistema. Extensão e substituição são possibilidades facilitadas por essas estruturas, mas não foram comprovadas com novo motor de layout ou nova fonte. Não foram medidos ganhos de desempenho, tempo de manutenção ou índices quantitativos de coesão.

No Strategy, o roteador de downloads passou de 244 para 67 linhas. A lógica de cada cliente foi movida para download-strategies.ts (103 linhas), onde cada integração é representada por uma entrada no mapa downloadStrategies. Acrescentar um novo cliente não requer modificar download.ts: basta adicionar uma entrada no mapa. A função createTorrentStrategy eliminou a repetição entre os três clientes de torrent, que antes tinham estrutura idêntica exceto pelo construtor e pelas opções.

No Adapter, o roteador de Usenet passou de 394 para 194 linhas. A construção do cliente NZBGet, que se repetia em cinco procedimentos, foi centralizada em createNzbgetClient dentro do adaptador. A conversão de callbacks para Promise, antes duplicada, foi encapsulada em promisify. SABnzbd e NZBGet, que possuem APIs incompatíveis, passaram a ser acessados pela mesma interface nos procedimentos do roteador e nas estratégias de download, favorecendo a substituição e extensão sem modificação dos clientes.

A combinação de Strategy e Adapter permitiu que as estratégias de Usenet em download-strategies.ts reutilizassem diretamente os adaptadores criados para usenet/router.ts, sem duplicação. Essa reutilização concreta demonstra o potencial de composição entre os dois padrões: o Adapter padroniza a interface dos clientes externos e o Strategy utiliza essa interface sem depender das implementações.

6. Conclusão

Facade concentrou operações antes distribuídas e reduziu a exposição de GridStack aos clientes. A ampliação de Template Method compartilhou o algoritmo de consulta das fontes remotas de ícones, preservando suas transformações específicas e a separação da fonte local.

Os testes de caracterização fornecem evidência de preservação dos cenários avaliados. No layout, houve validação interativa controlada com salvamento e recarregamento. Nos ícones, a rota real respondeu, mas seleção e preview ainda não foram comprovados na interface. Problemas globais anteriores de tipos e lint foram discriminados para não serem atribuídos às refatorações.

A decisão de rejeitar Command reforça que um padrão deve responder a uma necessidade identificada. Reuso entre emissores e duplicação, isoladamente, não obrigam uma solução com objetos de comando. Para Template Method, a discussão da base remota permitiu compartilhar a sequência necessária sem impor HTTP ao fluxo local.

As quatro refatorações realizadas — Facade, Template Method, Strategy e Adapter — partiram de problemas concretos identificados no código: integração distribuída com GridStack, repetição do fluxo HTTP entre fontes de ícones, acoplamento de clientes de download ao roteador, e duplicação de inicialização de clientes Usenet. Em cada caso, o padrão aplicado respondeu a uma necessidade específica, e a relação problema → padrão → refatoração → benefício foi documentada com trechos e métricas reais.

Entre as dificuldades encontradas, destaca-se a necessidade de compreender o comportamento preexistente antes de qualquer alteração — o que motivou a criação de testes de caracterização executados antes das refatorações. A divergência de geometria no navegador, registrada antes do início das modificações, foi documentada como limitação preexistente para não ser confundida com efeito colateral das refatorações. O Command foi considerado e rejeitado por não responder a um problema concreto, reforçando que a aplicação de padrões deve ser justificada tecnicamente.

Os principais aprendizados foram: a importância dos testes de caracterização como linha de base antes de refatorar código existente; a distinção entre duplicação legítima e duplicação acidental; e a possibilidade de compor padrões — como Strategy consumindo Adapter — para obter reutilização entre contextos distintos sem introduzir acoplamento adicional.

6.1 Participação da inteligência artificial

O ChatGPT/Codex apoiou a identificação de problemas, discussão de alternativas, planejamento das mudanças e organização das evidências. A ferramenta também ajudou na produção dos testes de caracterização e implementação dos trechos das refatorações.

7. Referências

GAMMA, Erich et al. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
FOWLER, Martin. Refactoring: Improving the Design of Existing Code. 2. ed. Addison-Wesley, 2018.
FEATHERS, Michael C. Working Effectively with Legacy Code. Prentice Hall, 2004.
ajnart. Homarr — A simple, yet powerful dashboard for your server. Repositório GitHub. Disponível em: https://github.com/ajnart/homarr. Acesso em: out. 2026.
odanielchristopher. reuso-trabalho-I — Repositório do trabalho de refatoração. GitHub. Disponível em: https://github.com/odanielchristopher/reuso-trabalho-I. Acesso em: out. 2026.
SOURCEMAKING. Design Patterns. Disponível em: https://sourcemaking.com/design_patterns. Acesso em: out. 2026.
REFACTORING.GURU. Design Patterns. Disponível em: https://refactoring.guru/design-patterns. Acesso em: out. 2026.
VITEST. Vitest — A Vite-native unit test framework. Documentação oficial. Disponível em: https://vitest.dev. Acesso em: out. 2026.